GUIA_ENTRADA_ESTANDAR_CPP

Guía de entrada estándar en C++ (incluyendo casos difíciles)

Volver al README principal

1) Reglas básicas de extracción

- `std::cin >> x`:
  - Lee tokens separados por espacios/saltos de línea.
  - Sirve para números y palabras simples.
  - No lee espacios internos en strings.
- `std::getline cin, s`:
  - Lee la línea completa (incluye espacios).
  - Ideal para textos largos o líneas mixtas.

2) Mezclar `>>` y `getline` sin errores

Problema típico:

- Lees un número con `cin >> n`.
- Luego llamas `getline` y recibes cadena vacía.

Solución:

```
int n;
std::cin >> n;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::string linea;
std::getline(std::cin, linea);
```

3) Lectura de líneas con comas, comillas y espacios

Caso A: línea completa cruda

```
std::string linea;
std::getline(std::cin, linea);
```

Ejemplo leído correctamente:
`"hola, mundo", 123, "a b c"`

Caso B: parsear CSV simple (comas dentro de comillas y comillas escapadas `""`)

```
#include <vector>
#include <string>

std::vector<std::string> splitCSVSimple(const std::string& s) {
    std::vector<std::string> out;
    std::string current;
    bool inQuotes = false;

    for (std::size_t i = 0; i < s.size(); ++i) {
        char ch = s[i];

        if (ch == '"') {
            if (inQuotes && i + 1 < s.size() && s[i + 1] == '"') {
```

```
                current.push_back('"'); // comilla escapada dentro de campo quoted
                ++i;
            } else {
                inQuotes = !inQuotes; // abre/cierra comillas
            }
        } else if (ch == ',' && !inQuotes) {
            out.push_back(current);
            current.clear();
        } else {
            current.push_back(ch);
        }
    }
    out.push_back(current);
    return out;
}
```

Notas:

- Soporta comas dentro de comillas: `"a,b",c` -> `a,b` y `c`.
- Soporta comillas escapadas tipo CSV: `"hola ""mundo"""` -> `hola "mundo"`.
- No valida errores de formato (por ejemplo, comillas sin cerrar).

4) Parsear números de una línea

```
std::string linea;
std::getline(std::cin, linea);
std::stringstream ss(linea);
int x;
while (ss >> x) {
    // procesar x
}
```

5) Leer todo el archivo de entrada estándar

```
std::string contenido(
    (std::istreambuf_iterator<char>(std::cin)),
    std::istreambuf_iterator<char>()
);
```

6) Buenas prácticas para evaluaciones con archivo por stdin

- No mezclar prompts (`"Ingrese..."`) con salida evaluable.
- Validar formato y reportar errores a `std::cerr`.
- Mantener salida limpia en `std::cout`.
- Si hay formatos mixtos, leer por línea (`getline`) y parsear después.

7) Patrón robusto recomendado

1. Leer líneas completas (`getline`).
2. Validar si la línea está vacía.
3. Parsear con `stringstream` o parser propio.
4. Emitir resultados en formato fijo.

Este patrón evita la mayoría de errores con espacios, comas y comillas.

9) Funciones de `string` usadas en este código (con ejemplos)

`size()`

- Devuelve la longitud del string.
- Uso en el código: controlar límites en loops y accesos (`i < s.size()`).
- Ejemplo:

```
string s = "hola";
size_t n = s.size(); // 4
```

`operator[]`

- Accede a un carácter por índice (sin chequeo de rango).
- Uso en el código: leer `s[i]` y `s[i + 1]` en el parser CSV.
- Ejemplo:

```
string s = "abc";
char c = s[1]; // 'b'
```

`substr(pos, len)`

- Extrae una subcadena desde `pos` con longitud `len`.
- Uso en el código: `trim` devuelve solo la porción útil.
- Ejemplo:

```
string s = "programacion";
string sub = s.substr(0, 4); // "prog"
```

`find(sub, posInicial)`

- Busca la primera ocurrencia de `sub` desde una posición inicial.
- Devuelve `string::npos` si no encuentra.
- Uso en el código: `replaceAll` itera todas las ocurrencias.
- Ejemplo:

```
string s = "a,b,c";
size_t p = s.find(",", 0); // 1
```

`replace(pos, len, nuevoTexto)`

- Reemplaza una porción del string.
- Uso en el código: convertir comillas UTF-8 a ASCII.
- Ejemplo:

```
string s = "hola mundo";
s.replace(5, 5, "C++"); // "hola C++"
```

`push_back(ch)`

- Agrega un carácter al final.
- Uso en el código: construir `current` carácter por carácter.
- Ejemplo:

```
string s;
s.push_back('A'); // "A"
```

`clear()`

- Vacía el contenido del string.
- Uso en el código: reiniciar el acumulador al cerrar un campo CSV.

- Ejemplo:

```
string s = "dato";
s.clear(); // ""
```

`string::npos`

- Constante que indica “no encontrado”.
- Uso en el código: condición de corte en `find` dentro de `replaceAll`.
- Ejemplo:

```
if (s.find("x") == string::npos) {
        // no existe "x"
}
```

10) Otras funciones de `string` útiles para parciales/talleres

`empty()`

- Verifica si el string está vacío.

```
if (linea.empty()) { /* línea vacía */ }
```

`compare()`

- Compara strings de forma explícita.

```
if (a.compare(b) == 0) { /* iguales */ }
```

`erase(pos, len)`

- Elimina un tramo de caracteres.

```
string s = "abcXYZdef";
s.erase(3, 3); // "abcdef"
```

`insert(pos, texto)`

- Inserta texto en una posición.

```
string s = "ac";
s.insert(1, "b"); // "abc"
```

`find_first_not_of(chars)` / `find_last_not_of(chars)`

- Muy útiles para hacer variantes de `trim`.

```
string s = "   hola  ";
size_t ini = s.find_first_not_of(" \t\n\r");
size_t fin = s.find_last_not_of(" \t\n\r");
```

`c_str()`

- Obtiene un `const char*` cuando una API antigua lo pide.

```
const char* p = s.c_str();
```

`getline(cin, s)` (de `<string>`)

- Aunque no es método miembro, es clave para entradas complejas con espacios.

```
string linea;
```

```
getline(cin, linea);
```

11) Cómo evitar un salto de línea adicional al final de la salida

En evaluadores automáticos, un `\n` extra al final puede hacer que la salida no coincida exactamente.

Problema típico

Imprimir siempre `"\n"` en cada iteración:

```
for (int i = 1; i <= nombres.getLong(); ++i) {
    cout << nombres.consultar(i) << "\n";
}
```

Eso deja una línea vacía extra al final del archivo si esa fue la última impresión global.

Patrón recomendado

Imprimir el salto **antes** de cada línea excepto la primera línea total de salida:

```
bool firstLine = true;

if (!firstLine) cout << "\n";
cout << "Caso " << caso << ": n=" << n << ", a=" << a << ", b=" << b;
firstLine = false;

for (int i = 1; i <= nombres.getLong(); ++i) {
    cout << "\n" << nombres.consultar(i);
}
```

o por ejemplo

```
for (int i = 1; i <= n; ++i) {
        if (i > 1) cout << "\n";
        cout << "Caso " << i << "  K=" << listaK.consultar(i)
            << "  Nombres=" << casosNombres.consultar(i);
    }
```

Regla práctica

- Si el formato exige coincidencia exacta, evitá terminar la última línea con `\n`.
- Preferí: separadores entre líneas (`"\n"` antes de imprimir la siguiente) en vez de `"\n"` al final de cada `cout`.
- Verificá con redirección: `make run-file` y revisá el conteo de líneas esperado.

12) Entrada estandar de Taller 2-2025

2
4 5 2
“alvaro espinoza”, “tobias”, “lord Hansell Campbell”, “mariana”
2 3 2
“alvaro espinoza”, “jimmy hendrix”, “slash”, “steven tyler”

¿Qué hace este código?

- Lee la cantidad de casos `t` desde la primera línea.
- Por cada caso, consume exactamente 2 líneas:
  - una línea numérica con 3 enteros (`n a b`),
  - una línea CSV con nombres entre comillas.

- Convierte comillas tipográficas UTF-8 (`“` y `”`) a comillas ASCII (`"`) para poder parsear de forma uniforme.
- Separa la línea CSV respetando:
  - comas dentro de campos entre comillas,
  - comillas escapadas con `""`.
- Guarda los nombres en `Lista<string>` (sin arreglos ni `vector`), y luego los imprime.

Detalles técnicos del formato de entrada

1. **Primera línea global**: un entero `t` (cantidad de casos).
2. **Por cada caso**:
   - **Línea 1 del caso**: tres enteros separados por espacio (`n a b`).
   - **Línea 2 del caso**: lista de nombres separada por comas.
3. **Comillas en nombres**:
   - Pueden venir como comillas ASCII (`"nombre"`) o tipográficas (`“nombre”`).
   - El parser normaliza tipográficas a ASCII antes de separar campos.
4. **Separación de campos**:
   - Solo corta por coma cuando está fuera de comillas.
   - Si hay `""` dentro de un campo quoted, lo interpreta como comilla literal.
5. **Espacios**:
   - Cada campo se recorta con `trim` para eliminar espacios al inicio/fin.
6. **Lectura robusta**:
   - Se usa `cin >> t` y luego `cin.ignore(...)` para evitar que el primer `getline` quede vacío.
   - El patrón `getline + stringstream` evita errores por mezcla de formatos.

Función de cada helper del código

- `trim`: elimina espacios laterales de cada campo parseado.
- `replaceAll`: reemplaza todas las ocurrencias de un substring.
- `normalizeQuotes`: convierte comillas tipográficas UTF-8 a `"`.
- `splitCSVSimpleToLista`: parser CSV simple con soporte de comillas y `""`.

Explicación línea por línea

trim (limpiar espacios en bordes)

- Entrada: recibe un string s.
- Paso 1: crea i en 0 y j en s.size().
- Paso 2: while (i < j && isspace(...s[i])) ++i;
  - Avanza i hasta el primer carácter que no sea espacio/tab/salto.
- Paso 3: while (j > i && isspace(...s[j - 1])) --j;
  - Retrocede j desde el final hasta el último carácter no-espacio.
- Paso 4: return s.substr(i, j - i);
  - Devuelve solo el segmento útil, sin espacios laterales.
- Resultado: " hola " pasa a "hola".

replaceAll (reemplazo global de subcadenas)

- Entrada: s (por referencia), from, to.
- Paso 1: pos = 0.
- Paso 2: while ((pos = s.find(from, pos)) != npos)
  - Busca desde pos la siguiente ocurrencia de from.
- Paso 3: s.replace(pos, from.size(), to);
  - Reemplaza esa ocurrencia por to.
- Paso 4: pos += to.size();

    - Mueve el cursor para seguir buscando después del reemplazo.
- Resultado: reemplaza todas las ocurrencias, no solo la primera.

normalizeQuotes (normalizar comillas tipográficas)

- Entrada: copia de string s.
- Paso 1: replaceAll(s, "\xE2\x80\x9C", """);
    - Convierte comilla izquierda tipográfica “ a comilla ASCII ".
- Paso 2: replaceAll(s, "\xE2\x80\x9D", """);
    - Convierte comilla derecha tipográfica ” a comilla ASCII ".
- Paso 3: return s.
- Resultado: deja una sola convención de comillas para que el parser CSV no falle.

splitCSVSimpleToLista (parser CSV simple a Lista)

- Entrada: string s (línea completa CSV).
- Paso 1: s = normalizeQuotes(s);
    - Unifica comillas para parseo consistente.
- Paso 2: crea out (Lista), current (campo en construcción), inQuotes = false.
- Paso 3: for sobre cada carácter ch de s.
- Paso 4: si ch es comilla ":
    - Si inQuotes es true y el siguiente char también es ":
        - current.push_back('"'); (comilla escapada CSV: "")
        - ++i para saltar la segunda comilla.
    - Si no, alterna estado inQuotes (abre/cierra campo quoted).
- Paso 5: si ch es coma y NO estás dentro de comillas:
    - Cierra campo actual: out.insertar(trim(current), out.getLong() + 1);
    - Limpia current.
- Paso 6: en cualquier otro caso:
    - Agrega ch a current.
- Paso 7: al terminar el for, inserta el último campo (también con trim).
- Paso 8: return out.
- Resultado:
    - Respeta comas internas en campos quoted.
    - Interpreta "" como comilla literal.
    - Entrega Lista sin usar vector/arreglos.

main (flujo completo de lectura)

- Paso 1: declara int t.
- Paso 2: cin >> t;
    - Lee cantidad de casos.
- Paso 3: cin.ignore `max(), '\n'` ;
    - Limpia el salto pendiente para que getline no lea vacío.
- Paso 4: for caso = 1 hasta t:
    - Declara lineaNums y lineaNombres.
    - getline(cin, lineaNums); (ejemplo: 4 5 2)
    - getline(cin, lineaNombres); (ejemplo CSV con nombres)
- Paso 5: parsea la línea numérica:
    - int n, a, b;
    - stringstream ss(lineaNums);
    - ss >> n >> a >> b;

- Paso 6: parsea nombres:
  - Lista nombres = splitCSVSimpleToLista(lineaNombres);
- Paso 7: imprime datos del caso y recorre nombres:
  - for i desde 1 hasta nombres.getLong()
  - nombres.consultar(i)
- Paso 8: termina con return 0.

Resumen del contrato de entrada

- Primera línea: t.
- Por cada caso: 2 líneas exactas.
  - Línea 1: 3 enteros.
  - Línea 2: CSV de nombres (con comillas ASCII o tipográficas).
- El parser:
  - normaliza comillas,
  - separa por coma fuera de comillas,
  - conserva comas internas quoted,
  - y hace trim de bordes.

```
#include <iostream>
#include <string>
#include <sstream>
#include <limits>
#include <cctype>
#include "Lista.hpp"

using namespace std;

static string trim(const string& s) {
    size_t i = 0, j = s.size();
    while (i < j && isspace((unsigned char)s[i])) ++i;
    while (j > i && isspace((unsigned char)s[j - 1])) --j;
    return s.substr(i, j - i);
}

static void replaceAll(string& s, const string& from, const string& to) {
    size_t pos = 0;
    while ((pos = s.find(from, pos)) != string::npos) {
        s.replace(pos, from.size(), to);
        pos += to.size();
    }
}

static string normalizeQuotes(string s) {
    replaceAll(s, "\xE2\x80\x9C", "\""); // “
    replaceAll(s, "\xE2\x80\x9D", "\""); // ”
    return s;
}

static Lista<string> splitCSVSimpleToLista(string s) {
    s = normalizeQuotes(s);

    Lista<string> out;
    string current;
    bool inQuotes = false;

    for (size_t i = 0; i < s.size(); ++i) {
        char ch = s[i];
```

```
        if (ch == '"') {
            if (inQuotes && i + 1 < s.size() && s[i + 1] == '"') {
                current.push_back('"'); // comilla escapada ""
                ++i;
            } else {
                inQuotes = !inQuotes;
            }
        } else if (ch == ',' && !inQuotes) {
            out.insertar(trim(current), out.getLong() + 1);
            current.clear();
        } else {
            current.push_back(ch);
        }
    }

    out.insertar(trim(current), out.getLong() + 1);
    return out;
}

int main() {
    int t;
    cin >> t;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    for (int caso = 1; caso <= t; ++caso) {
        string lineaNums, lineaNombres;
        getline(cin, lineaNums);      // ej: 4 5 2
        getline(cin, lineaNombres);   // ej: “alvaro...”, “tobias”, ...

        int n, a, b;
        stringstream ss(lineaNums);
        ss >> n >> a >> b;

        Lista<string> nombres = splitCSVSimpleToLista(lineaNombres);

        // Ejemplo de uso (adaptá salida a tu consigna):
        cout << "Caso " << caso << ": n=" << n << ", a=" << a << ", b=" << b << "\n";
        for (int i = 1; i <= nombres.getLong(); ++i) {
            cout << nombres.consultar(i) << "\n";
        }
    }

    return 0;
}
```

13) Entrada estandar de Taller 1-2025

2
3
5
judas juan
iscariote simon
jacobo natanael
mateo felipe
pedro santiago
andres
tomas
jesus

Que hay que hacer?

1. Leer `n`
2. Leer los `n` valores `k`
3. Inicializar `n` listas vacías de nombres
4. Consumir el salto pendiente (ignore)
5. Mientras haya líneas (getline hasta EOF), parsear cada línea con stringstream
6. El primer token va a la lista 1, el segundo a la 2, etc.

```
#include <iostream>
#include <string>
#include <sstream>
#include <limits>
#include "../include/Lista.hpp"

using namespace std;

int main() {
    int n;
    cin >> n;

    Lista<int> listaK;
    for (int i = 0; i < n; ++i) {
        int k;
        cin >> k;
        listaK.insertar(k, listaK.getLong() + 1);
    }

    Lista<Lista<string> > casosNombres;
    for (int i = 0; i < n; ++i) {
        Lista<string> vacia;
        casosNombres.insertar(vacia, casosNombres.getLong() + 1);
    }

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    string linea;
    while (getline(cin, linea)) {
        if (linea.size() == 0) continue;

        stringstream ss(linea);
        string nombre;
        int col = 1; // columna 1..n

        while (ss >> nombre && col <= n) {
            Lista<string> aux = casosNombres.consultar(col);
            aux.insertar(nombre, aux.getLong() + 1);
            casosNombres.modificar(col, aux);
            ++col;
        }
    }

    // Debug de verificación
    for (int i = 1; i <= n; ++i) {
        cout << "Caso " << i << "  K=" << listaK.consultar(i)
             << "  Nombres=" << casosNombres.consultar(i) << endl;
    }

    return 0;
}
```

14) Entrada estandar de Taller 1-2024 CSV Moda

Entrada esperada

```
Marca,Precio
ford,4
ford,3
ford,4
chevrolet,4
```

Cómo se entiende esta entrada

- La primera línea es especial: no trae datos de la tabla, trae los nombres de las columnas.
- Cada línea siguiente representa una fila del CSV.
- Cada fila tiene la misma cantidad de campos que el encabezado.
- Los campos están separados por coma.
- En esta consigna no hay espacios después de la coma y no hay celdas vacías.

Qué hay que leer primero

1. Leer la primera línea completa con `getline`.
2. Separar esa línea por comas para obtener los encabezados.
3. Crear una estructura por columna, no por fila, porque luego debes calcular la moda de cada columna completa.

Por qué conviene guardar por columnas

Si guardas por filas, después tendrías que volver a recorrer toda la tabla para reconstruir cada columna.
Si guardas por columnas desde el inicio:

- la columna 1 acumula todos los valores de `Marca`
- la columna 2 acumula todos los valores de `Precio`
- luego puedes calcular la moda de cada columna directamente

Lectura paso a paso del `main`

Supón esta entrada:

```
Marca,Precio
ford,4
ford,3
chevrolet,4
```

Paso 1: leer el encabezado completo

```
string linea;
getline(cin, linea);
```

Después de esto, `linea` vale:

```
Marca,Precio
```

Paso 2: separar el encabezado por comas

```
Lista<string> encabezados;
separarCSV(linea, encabezados);
```

Ahora `encabezados` contiene:

- posición 1: `Marca`
- posición 2: `Precio`

Paso 3: crear una lista de columnas vacías

```
Lista<Lista<string> > columnas;
for (int i = 1; i <= encabezados.getLong(); ++i) {
    columnas.insertar(Lista<string>(), columnas.getLong() + 1);
}
```

Aquí se crean dos columnas vacías:

- `columnas[1]` para todos los valores de `Marca`
- `columnas[2]` para todos los valores de `Precio`

Paso 4: leer cada fila restante con `getline`

```
while (getline(cin, linea)) {
    Lista<string> fila;
    separarCSV(linea, fila);
    ...
}
```

Cada iteración toma una fila completa del CSV. Por ejemplo:

- primera iteración: `ford,4`
- segunda iteración: `ford,3`
- tercera iteración: `chevrolet,4`

Paso 5: separar la fila en celdas

Si `linea` es `ford,4`, entonces:

```
Lista<string> fila;
separarCSV(linea, fila);
```

deja:

- `fila[1] = ford`
- `fila[2] = 4`

Paso 6: mover cada celda a su columna correspondiente

```
for (int i = 1; i <= encabezados.getLong(); ++i) {
    columnas.consultarRef(i).insertar(fila.consultar(i), columnas.consultarRef(i).getLong() + 1);
}
```

Después de procesar las tres filas del ejemplo, queda:

- columna `Marca`: `ford`, `ford`, `chevrolet`
- columna `Precio`: `4`, `3`, `4`

Cómo se calcula la moda después de leer

Ya con las columnas completas, solo recorres cada una:

```
Lista<string> modas;
for (int i = 1; i <= columnas.getLong(); ++i) {
    modas.insertar(calcularModa(columnas.consultar(i)), modas.getLong() + 1);
}
```

Entonces:

- la moda de `Marca` es `ford`
- la moda de `Precio` es `4`

Qué imprime el programa al final

Primero imprime el encabezado original:

```
Marca,Precio
```

Luego imprime la fila con las modas:

```
ford,4
```

Resumen mental rápido para este ejercicio

1. `getline` para leer una línea completa del CSV.
2. `separarCSV` para cortar esa línea por comas.
3. Guardar datos por columnas.
4. Calcular una moda por cada columna.
5. Imprimir dos filas: encabezados y modas.

15) Metodos `moda()` y `mediana()` en la clase Lista

Se añadieron dos métodos útiles a Programacion/include/Lista.hpp:

- `moda()`
- `mediana()`

`moda()`

Uso:

```
Lista<int> datos;
datos.insertar(4, 1);
datos.insertar(2, 2);
datos.insertar(4, 3);

int m = datos.moda();
```

Comportamiento:

- Cuenta cuántas veces aparece cada valor.
- Si existe una única frecuencia máxima, devuelve ese valor.
- Si la lista está vacía, lanza excepción.
- Si no hay moda única, también lanza excepción.

Ejemplo:

```
Lista<string> nombres;
nombres.insertar("ana", 1);
```

```
nombres.insertar("luis", 2);
nombres.insertar("ana", 3);

string moda = nombres.moda(); // "ana"
```

`mediana()`

Uso:

```
Lista<int> datos;
datos.insertar(7, 1);
datos.insertar(1, 2);
datos.insertar(9, 3);

double med = datos.mediana();
```

Comportamiento:

- Hace una copia de la lista.
- Ordena la copia de menor a mayor.
- Si la cantidad de elementos es impar, devuelve el del centro.
- Si es par, devuelve el promedio de los dos centrales.
- Si la lista está vacía, lanza excepción.
- Solo está disponible para tipos numéricos.

Ejemplos:

```
Lista<int> a;
a.insertar(7, 1);
a.insertar(1, 2);
a.insertar(9, 3);
double medA = a.mediana(); // 7

Lista<int> b;
b.insertar(10, 1);
b.insertar(4, 2);
b.insertar(6, 3);
b.insertar(8, 4);
double medB = b.mediana(); // 7
```

Cuándo conviene usarlos

- Usa `moda()` cuando necesites el valor más repetido.
- Usa `mediana()` cuando necesites el valor central de un conjunto numérico ordenado.
- Si tu consigna dice que en caso de empate debes imprimir `ninguna`, captura la excepción de `moda()` y tradúcela a la salida pedida.